

Voorblad

Naam: Lukas Bellaart

Studentnummer: 1099432

Vak: Informatica

Docent 1: Ditmar Commandeur

Docent 2: Lotte Muilwijk

Datum: 02/11/2024

Voorwoord

Beste lezer,

Bij deze presenteer ik mijn dossier, waarin ik mijn ontwikkeling en prestaties van het afgelopen semester tijdens Basecamp heb vastgelegd. Ik hoop dat jullie door het lezen van dit dossier een goed beeld krijgen van mijn progressie als student.

Ik vond BaseCamp een leuke ervaring, waarbij ik veel heb geleerd, niet alleen op technisch gebied, maar ook qua professionele vaardigheden. Ik ging dit semester met plezier met school en keek uit naar de lessen. De docenten waren prettig in omgang en konden de lesstof heel fijn brengen. Daarom wil ik mijn docenten, Ditmar Commandeur en Lotte Muilwijk, bedanken voor hun inzet en begeleiding.

Verder wil ik graag mijn leerteam, met name Gülseren, bedanken voor de fijne samenwerking. En ook Manasseh, mijn collega die vroegtijdig met de opleiding gestopt is, wil ik bedanken voor het samenwerken aan de game die tijdens de challenge weeks gemaakt moest worden, maar met name voor de gezellige culinaire uitjes naar de Markthal.

Ik kijk uit naar het volgende semester!

Lukas Bellaart, 1 januari 2025

Inhoudsopgave

1. Evaluatiecyclus

- 1.1 Zelfevaluatie - week 3
- 1.2 Feedbacksessie: verslag - week 4/5
- 1.3 Plan van aanpak: leerdoelen - week 6
- 1.5 Zelfevaluatie - week 8
- 1.6 Feedbacksessie: verslag - week 9/10
- 1.7 Plan van aanpak: leerdoelen - week 11
- 1.8 Peerevaluatie - week 12

1.4 F



2. Ondersteunend bewijs

- 2.1 Supporting topic week 2 – Data formats
- 2.2 Supporting topic week 6 – Debugging
- 2.3 Supporting topic week 7 – Unit testen
- 2.4 Supporting topic week 10 – Exception handling

3. Bijlagen

- 3.1 Overige bijlagen die bijdragen aan je leeruitkomsten (optioneel)

1. Evaluatiecyclus

1.1 Zelfevaluatie (week 3)

- **Hoe denk jij na de eerste weken over je studiekeuze?**

Ik vind dat ik een goeie keuze heb gemaakt, omdat ik met plezier naar school ga. De hoeveelheid en snelheid van de stof valt tot nu toe nog mee.

- **Hoe ervaar je BaseCamp ten opzichte van je verwachtingen vooraf?**

Ik had verwacht dat vooral op deeltijd, BaseCamp heel snel zou gaan, maar dat valt gelukkig wel mee en kan ik het tempo goed bijbenen. Ook vind ik de docenten relaxter dan ik had verwacht.

- **Wat vind je van de klas?**

Een paar klasgenoten kende ik al voordat ik aan de opleiding begon, maar de rest van de klas vind ik ook wel gezellig, op een paar afleidende elementen na.

- **Wat vind je van je leerteam?**

Mijn leerteam is uniek omdat iedereen in een andere fase van het programmeren zit, maar daarom sluiten we goed op elkaar aan. Wij gaan vrijdag 4 oktober ook met z'n allen naar een event in de jaarbeurs in Utrecht.

- **Wat ging de afgelopen weken goed?**

De opdrachten kon ik zonder al te veel moeite maken. Het belangrijkste vind ik dat ik het goed kan vinden met mijn leerteam.

- **Wat ging de afgelopen weken minder goed? Of was moeilijk?**

Ik vond het rekenen met hexadecimale lastig, maar na meer uitleg lukte het. Het blijft voor mij wel een lastig onderwerp.

- **Wat is je stappenplan als je vastloopt?**

Mijn stappenplan is: eerst googelen, dan vragen aan mijn leerteam, vervolgens een collega in het tweede leerjaar vragen, daarna de peercoach en als laatste redmiddel Ditma.

- **Welke bronnen/hulpmiddelen heb jij tot je beschikking voor de BaseCamp?**

Ik heb het Python boek die ik kan gebruiken als bron, ook kan ik met het gebruik van Google Scholar veel wetenschappelijke bronnen vinden. Zo heb ik bijvoorbeeld toen we bezig waren met de supporting topic data formats, via Google Scholar deze bron gevonden: D. Hua, A. Weiland and S. Drummond, Bits, Bytes, and Binary: The Mathematics of Computers, Tech Directions, 2014.

Voor hulpmiddelen heb ik ook nog collega's die dezelfde stof gehad hebben waar ik naartoe kan met vragen.

- ***Wat vind je van de inhoud van BaseCamp?***

De stof die we krijgen in BaseCamp is de basis voor alles, maakt niet uit welke taal het is, de principes blijven zich herhalen. Daarom vind ik het een leuke en interessante inhoud.

- ***Wat vind je van het leren programmeren in Python?***

Python is een goede taal om mee te beginnen, omdat de syntax enorm op Engels lijkt.

Python kan worden gebruikt om kleine dingen te automatiseren, maar ook voor grootschalige AI.

Het is een goede allround programmeertaal, en daarom vind ik het een fijne taal om mee te beginnen.

Je hebt de checklist studiegewoonten en -vaardigheden ingevuld.

- ***Wat was de uitslag?***

De uitslag was 125 punten, dat staat voor “Je hebt gewerkt aan het ontwikkelen van een reeks vaardigheden en gewoontes die het zullen vergemakkelijken om academisch succes te behalen bij het hoger onderwijs. Bekijk de checklist om te zien waar je je werkwijzen moet aanscherpen om in de “‘bijna altijd’ categorieën terecht te komen. Als je eenmaal de specifieke werkwijzen hebt vastgesteld die moeten worden verbeterd, stel je jezelf gerichte studievaardigheidsdoelen.”

- ***In hoeverre herken je jezelf in de uitslag?***

Ik denk dat ik inderdaad heb gewerkt aan een manier om beter te kunnen studeren door te zorgen voor een rustige plek en veel betrokken te zijn in mijn studiegroep. Het is mij ook duidelijk aan welke punten ik moet werken om dit verder te verbeteren. Met name op het gebied van concentratie en motivatie. 

1.2 Feedbacksessie: verslag - week 4/5

1. Welke zaken doe je goed volgens je docent?

Volgens Ditmar lig ik op schema met de leerstof en doe ik actief mee in de les, vooral binnen mijn groepje. Lotte gaf aan dat zij het idee heeft dat ik qua niveau goed op mijn plek zit en dat ik prima pas binnen het HBO.

2. Zijn er verbeterpunten besproken? Zo ja, welke?

Ja, er zijn een paar verbeterpunten besproken. Lotte gaf aan dat ik af en toe de neiging heb om af te dwalen tijdens de les. Ze adviseerde dat ik, wanneer dat gebeurt, het beste kan vragen waar we gebleven zijn of dit kan aangeven.

3. Hebben jij en je docent bepaalde afspraken gemaakt voor de komende tijd? Zo ja, welke?

Nee, we hebben voor de komende tijd geen afspraken gemaakt.

4. Wat vind je zelf van de ontvangen feedback? Herken je wat er besproken is?

Ja, ik herken mezelf in de feedback. Het afdwalen tijdens de les gebeurt soms. Nu het tempo nog wat lager ligt, heeft dat nog niet veel invloed (al is het natuurlijk niet ideaal). Maar zodra het tempo hoger ligt, kan het echt een probleem worden als ik blijf afdwalen, dus ik moet daar zeker op letten en vragen als ik iets heb gemist.

5. Welke zaken uit het gesprek kun je meenemen ter verbetering in de toekomst (plan van aanpak)?

- Ik ga proberen te voorkomen dat mijn aandacht afdwaalt door aantekeningen te maken tijdens de lessen. Ook helpt het naar de docent te kijken wanneer deze iets vertelt in plaats van naar de PowerPoint op het bord. Als ik iets heb gemist, zal ik het meteen aangeven of vragen of de docent het kan herhalen, zodat ik niets belangrijks mis.

- Hoewel dit punt niet specifiek uit het gesprek kwam, denk ik dat het kan helpen om mijn focus te verbeteren. Door actiever mee te doen in de les, bijvoorbeeld door vragen te stellen of mee te denken tijdens discussies, kan ik mijn aandacht beter vasthouden en het afdwalen verminderen.

1.3 Plan van aanpak: leerdoelen - week 6

1. Leerdoel Professional Skills

Specifiek	Het probleem is dat ik soms afdwaal in de les. Ik wil werken aan concentratie in de les daarom ga ik tijdens elke les aantekeningen maken zodat ik mijn focus kan houden en de stof beter kan verwerken. Na de les op maandag maak ik er thuis een samenvatting in Word van die ik deel met mijn leerteam in teams.
Meetbaar	Na elke les zijn er aantekeningen gemaakt die ingeleverd kunnen worden in het dossier, die upload ik aan het einde van de dag in ons leerteamkanaal in teams.
Acceptabel	Ik wil gefocust blijven tijdens de les en denk dat het maken van aantekeningen een goede manier is om dit te bereiken. Het vereist discipline om consequent aantekeningen te maken, en samenwerking omdat ik het met mijn leerteam deel.
Realistisch	Ik hoop dat het sturen van mijn aantekeningen naar mijn leerteam na elke les mij zal motiveren om deze gewoonte vol te houden.
Tijd	Dit leerdoel ga ik in arch 2 mee beginnen en ik zorg ervoor dat elke maandag minimaal één aantekening van de les is gemaakt

In arch 2 wil ik tijdens elke les maandag aantekeningen maken om mijn concentratie erbij te houden. Na de les maak ik er een samenvatting van en deel ik deze naar mijn leerteam.

2. Leerdoel programmeren

Specifiek	Ik wil de nieuw geleerde technieken zoals bijvoorbeeld list comprehensions toepassen op oudere projecten uit BaseCamp. Op deze manier kan ik het verschil zien tussen de verschillende manieren om een opdracht op te lossen.
Meetbaar	Het doel is om elke vrijdag één uur te spenderen om een oude assignment uit BaseCamp kritisch te bekijken waar ik nieuw geleerde technieken kan gebruiken en de code eventueel kan verbeteren. Ik houd bij welke assignments ik heb aangepast.
Acceptabel	Dit doel helpt mij te reflecteren op eerder geschreven code en helpt mij om mijn eerdere werk te verbeteren. Dit zou moeten voorkomen dat ik eventuele gemaakte fouten herhaal in de toekomst.
Realistisch	Het is haalbaar om één opdracht per week te herzien, aangezien ik op vrijdag hiervoor één uur tijd vrijmaak.
Tijd	Ik begin in arch 2 en elke vrijdag spendeer ik één uur om een oude opdracht te herzien en deze aan te passen met nieuwere technieken waar nodig, zodat ik tegen het einde van de arch een lijst heb met aangepaste opdrachten.

In arch 2 wil ik elke vrijdag één uur lang spenderen om een eerder gemaakte opdracht uit BaseCamp kritisch te bekijken om te zien of deze kan worden verbeterd door het toepassen van de nieuw geleerde technieken.

Deel A Peerevaluatie

1. Vul hieronder een aantal zaken waar je rekening mee moet houden bij het geven of ontvangen van feedback [1] [2]. 

Wel doen:

- Concrete feedback geven
- Eventuele voorbeelden

Niet doen:

- Feedback geven wat onhaalbaar is.
- Onduidelijke feedback.

2. Geef ieder leerteamlid feedback (positieve punten en/of verbeterpunten), rekening houdend met de antwoorden bij vraag 1.

Teamlid 1 feedback Salwan:

Salwan is goed in het accepteren van feedback, wel heeft hij moeite met het snappen/oplossen van errors, hij kan beter eerst Googlen wat de error is.

Teamlid 2 feedback Gülseren:

Gülseren is altijd op tijd aanwezig, dat is fijn omdat zij goed is in tekstuele elementen van de opleiding.

Tijdens de zelfwerkuren in de klas kan zij beter ergens alleen zitten omdat ze moeite heeft met concentreren.

3. Bepreek de feedback van vraag 2 met je leerteamleden. Noteer hier welke feedback je hebt ontvangen.

Ontvangen feedback Teamlid 1 Salwan: Salwan vindt mij een behulpzaam en verantwoordelijk persoon.

Ontvangen feedback Teamlid 2 Gülseren: Gülseren vindt mij de meest ervaren in het team, en fijn dat ik wekelijks vragen stel of dat de opdrachten gelukt zijn of vragen zijn.

Als feedback kreeg ik te horen dat het handig is als ik aan mijn concentratie werk.

Deel B Code review

1. Welke programmeeropdracht van welke student heb je bekeken?

Programmeeropdracht: A2W5P1 - Automated arithmetics

Student: Manasseh Stam

2. Hebben de variabelen goede namen? (is het een omschrijving van hetgeen erin zit? Welke zou je anders noemen? Zijn er overbodige variabelen? Komt het overeen met hoe jij het hebt gedaan?)

De variabelen hebben goede namen. Er zijn variabelen die niet nodig zijn omdat er op deze plekken tuple packing gebruikt had kunnen worden.

Ook worden variabelen in een if statement toegekend die er beter buiten hadden kunnen staan om het overzichtelijker te maken.

Oplossing Manasseh:

```
def generate_exercise(arithmetic_type: str) -> tuple[str, int]:
    answer = generate_random_number()
    formula = ''

    if arithmetic_type == 'summation':
        first_number = generate_random_number()
        second_number = generate_random_number()
        formula = f'{first_number} + {second_number} = '
        answer = first_number + second_number
    elif arithmetic_type == 'multiplication':
        first_number = generate_random_number()
        second_number = generate_random_number()
        formula = f'{first_number} * {second_number} = '
        answer = first_number * second_number
    elif arithmetic_type == 'subtraction':
        first_number = generate_random_number()
        second_number = generate_random_number()
        formula = f'{first_number} - {second_number} = '
        answer = first_number - second_number
```

Verbeterde oplossing:

```
def generate_exercise(arithmetic_type: str) -> tuple[str, int]:
    first_number = generate_random_number()
    second_number = generate_random_number()
    if arithmetic_type == "summation":
        return f"{first_number} + {second_number} = ", first_number + second_number
    elif arithmetic_type == "multiplication":
        return f"{first_number} * {second_number} = ", first_number * second_number
    elif arithmetic_type == "subtraction":
        return f"{first_number} - {second_number} = ", first_number - second_number
```

Verder komt het in grote lijnen overeen met wat ik heb gedaan.

3. **Doet de code wat het moet doen? En is de code duidelijk in wat het doet? In hoeverre komt het overeen met hoe jij het hebt opgelost?**

De code werkt naar behoren en is duidelijk geschreven, de “main” function ziet er bij Manasseh netter uit dan bij mij omdat hij de validatie en het berekenen van hoeveel goede antwoorden in een andere functie doet.

4. **Hoe zien de if/while statements eruit? (denk aan: duidelijk/clean, zijn er overbodige statements?) In hoeverre komt het overeen met hoe jij het hebt opgelost?**

If statements zien er goed uit. Maar zoals ook al bij punt 2 vermeld worden er variabelen aangemaakt in sommige if statements, die er beter buiten hadden kunnen staan.

Apart van die punten liggen de oplossingen dicht bij elkaar.

Huidige oplossing:

```
if arithmetic_type == 'summation':
    first_number = generate_random_number()
    second_number = generate_random_number()
    formula = f'{first_number} + {second_number} = '
    answer = first_number + second_number
```

Verbeterde oplossing:

```
first_number = generate_random_number()
second_number = generate_random_number()
if arithmetic_type == 'summation':
    formula = f'{first_number} + {second_number} = '
    answer = first_number + second_number
```

5. **Zijn de PEP8 guidelines voor Python toegepast? Waar gaat dat goed? Waar kan het beter?**

PEP8 is toegepast, dit kan natuurlijk makkelijk gespot worden met visual studio code extensies.

6. **Kan de oplossing simpeler? Vergelijk de code van je medestudent met je eigen code van deze opdracht. Wat valt je op?**

De functie “generate_random_number” is overbodig aangezien de functie maar twee keer gebruikt wordt. Ook zitten er in de “generate_exercise” functie meer mogelijkheden om het simpeler te maken zoals:

- De random variabelen niet in de if statement genereren maar erbuiten.
- Tuple packing gebruiken in de return, dit voorkomt extra benodigde variabelen.

7. **Wordt eventuele verkeerde input afgevangen?**

Ja, er zijn twee functies om verkeerde input af te vangen.

Eén om te valideren of de rekenkundige bewerking: optellen, vermenigvuldiging of aftrekking is en één om te valideren of het antwoord dat gegeven is op de rekenvraag een nummer is.

```
arithmetic_type = get_valid_input(
    is_arithmetic_type_valid,
    "Enter arithmetic type (summation, multiplication, subtraction): ")
user_answer = get_valid_input(is_answer_valid, formula)
```

1.5 Zelfevaluatie - week 8

1. Hoe denk jij nu over je studiekeuze?

Mijn studiekeuze voelt nog steeds goed. Ik merk dat de stof overeenkomt met mijn interesses

2. Hoe ervaar je Basecamp ten opzichte van je verwachtingen vooraf?

Het is makkelijker dan verwacht

3. Wat ging de afgelopen weken goed? Of waar ben je trots op?

Het tempo ging afgelopen weken goed, ik heb alle assignments en problems op twee na afgekregen.

Het aantekeningen maken tijdens de les lukte goed, en ik merkte dat mijn concentratie daardoor verbeterde. Mijn gedachten dwaalden minder snel af omdat ik constant bezig was met de lesstof.

4. Wat ging de afgelopen weken minder goed? Of wat was moeilijk?



Ik had moeite met het goed begrijpen van de vraagstelling bij de assignment van week 7, dat maakte het lastig om de opdracht goed te maken.

Het direct uitwerken van de samenvatting is mij niet altijd gelukt omdat ik prioriteit stelde aan het uitwerken van de opdrachten van die week, hierdoor vergat ik soms de samenvatting te maken. Het gevolg was dat ik de samenvatting pas later in de week maakte wat lastig was omdat de stof iets was weggezakt.



5. In hoeverre past het schema van Basecamp bij jouw studietempo?

Goed, ik heb altijd de vrijdag vrij genomen voor eventuele schoolwerk dat ik maandag niet afkrijg, ik heb die dag nog niet al te veel moeten gebruiken

Evaluatie leerdoelen

6. Welke leerdoelen heb je opgesteld?

a. **Professional Skills:** Samenvattingen maken tijdens de les.

b. **Programmeren:** Oudere opdrachten opnieuw schrijven met nieuwe technieken.

7. Welke activiteiten heb je ondernomen voor het werken aan beide leerdoelen?

Ik heb voor de professionele skills elke week op maandag een samenvatting gemaakt van wat er in de les verteld is.

Voor het programmeren heb ik elke vrijdag een oude opdracht opnieuw geschreven om te reflecteren.

8. Wat zou je eventueel nog meer kunnen doen? Of anders kunnen doen?

Ik zou eventueel nog een samenvatting van de lesstof kunnen maken en die met mijn leerteam kunnen bespreken.



Ook kan ik samen met mijn leerteam naar de herschreven code kijken om daar feedback over te vragen en hun eventueel tips te geven.



1.6 Feedbacksessie: verslag - week 9/10

1 Welke zaken doe je goed volgens je docent?

Volgens Lotte ben ik een energieke, gemotiveerde vrolijke jongen.

Het was fijn dat ik binnen het leerteam de leider rol op heb gepakt.

In Codegrade ging het redelijk goed volgens Ditmar, ook al zijn de problems van week 7 nog niet af.

2 Zijn er verbeterpunten besproken? Zo ja, welke?

We hebben één verbeterpunt besproken, dat ging over het mentaal afhaken tijdens de les.

Hiervoor was de suggestie dat mijn leerteam mij aantikt als zij merken dat ik afdwaal.

3 Hebben jij en je docent bepaalde afspraken gemaakt voor de komende tijd? Zo ja, welke?

Er zijn geen afspraken gemaakt.

4 Wat vind je zelf van de ontvangen feedback? Herken je wat er besproken is?

De gekregen feedback vind ik kloppend, ik herken mezelf in wat er besproken is.

In de vorige feedbacksessie hadden we het ook over het afdwalen, naar mijn idee is het sindsdien minder.

5 Welke zaken uit het gesprek kun je meenemen in je plan van aanpak?

Mijn plan van aanpak na de vorige keer was om notities te maken tijdens de les om mijn aandacht erbij te houden. Naar mijn mening is dat gelukt dus dit is iets wat ik zal blijven doen. Er zijn in het gesprek verder geen zaken naar voren gekomen om mee te nemen in mijn plan van aanpak.

1.7 Plan van aanpak: leerdoelen - week 11

Mijn leerdoel professional skills:

Specifiek	Ik heb vindt het moeilijk om betrouwbare technische bronnen te vinden. Daarom ga ik één betrouwbare technische bron zoeken die bij de lesstof past per week.
Meetbaar	Per week ga ik op de vrijdag één uur spenderen om een betrouwbare technische bron te zoeken die past bij de lesstof die afgelopen maandag behandeld is. Als ik er één heb gevonden die betrouwbaar is, deel ik deze vervolgens met mijn leerteam. Dit herhaalt zich voor elke week in arch 3. Het resulteert in een lijst van onderwerpen en bronnen voor het dossier.
Acceptabel	Door dit te doen hoop ik dat het bronnen zoeken mij makkelijker afgaat in de toekomst en dat ik daar bij het dossier profijt van heb.
Realistisch	Het is haalbaar om hier elke week tijd voor vrij te maken. Door de bronnen ook binnen het leerteam te delen, vormen zij een stok achter de deur.
Tijd	Dit leerdoel herhaalt zich elke week in arch 3. Elke vrijdag spendeer ik hier één uur aan.

Ik wil in arch 3 oefenen met het vinden van betrouwbare technische bronnen. Ik ga elke vrijdag één uur besteden aan het zoeken van bronnen op basis van de op maandag behandelde lesstof.

Mijn leerdoel programmeren:

Specifiek	Ik pas het schrijven van unit tests op dit moment weinig toe. Ik wil hier meer mee oefenen, omdat unit tests handig zijn om code te testen en het in het vak ook veel gebruikt wordt.
Meetbaar	Ik ga op elke vrijdag één uur voor een probleem van de week unit tests schrijven. Hier gebruik ik de documentatie van Python unittest voor. De code hiervan zet ik als bewijslast in mijn dossier.
Acceptabel	Deze vaardigheden om unit tests te schrijven resulteert in beter werkende code en wordt veel gebruikt in het zakelijk leven. Dit motiveert mij om het goed aan te pakken.
Realistisch	Het is haalbaar om één unit test per week te schrijven. Hiervoor gebruik ik de documentatie van Python unittest.
Tijd	Ik ga hier elke vrijdag in arch 3 één uur voor zitten.

Ik wil in arch 3 vaker unit tests schrijven. Elke vrijdag ga ik één uur besteden aan unit tests schrijven voor één probleem van de die week behandelde lesstof.

1.8 Peerevaluatie - week 12

Deel A Peerevaluatie

1. Waarin kun jij anderen in je leerteam goed ondersteunen?

Ik ondersteun Gülseren met de programmeer opdrachten en probeer haar te helpen met het maken van een stappenplan om een opdracht te maken. Ik vraag elke week of de opdrachten in CodeGrade gelukt zijn, als dat niet zo is probeer ik tips te geven om op de oplossing te komen, in plaats van direct de vraag te beantwoorden. Elke week ben ik aanwezig, dit helpt iedereen met het krijgen motivatie.

2. Waar kunnen anderen in je leerteam jou mee helpen?

Ik vind het maken van het dossier lastig. Ik maak redelijk veel spelfouten en loop vaak vast met overzetten van mijn gedachten tot een kloppend stukje tekst.

We nemen samen onze dossiers door en tijdens het lezen geven we elkaar advies over eventuele aanpassingen of hoe de tekst beter verwoord kan worden. Elke woensdag zitten we in Teams, dit motiveert mij om aan mijn dossier te werken.

Deel B Code review

1. Zoek iemand uit een ander leerteam en kies samen een programmeeropdracht uit week 3, 5 of 6 om te reviewen. Bekijk de code van je medestudent en geef antwoord op de onderstaande vragen.

2. Welke programmeeropdracht van welke student heb je bekeken?

Programmeeropdracht: A3W10P2 - Python tail program

Student: Loïs Verheij

3. Hebben de variabelen goede namen? (Is het een omschrijving van hetgeen erin zit? Welke zou je anders noemen? Zijn er overbodige variabelen? Komt het overeen met hoe jij het hebt gedaan?)

De variabelen hebben een duidelijke naam en volgen de Python standaarden.

Er zijn geen overbodige variabelen.

Zelf heb ik meer variabelen, omdat ik functies heb gebruikt die het openen en uitlezen van het bestand scheiden van de code waar het geprint wordt.

```
fileName = sys.argv[1]
filePath = os.path.join(os.getcwd(), fileName)

if not validateFile(filePath):
    print("Error reading file:", fileName)
    return

fileLines = openFile(filePath, fileName, reverse=True)

for lineIndex in range(min(10, len(fileLines))):
    print(fileLines[lineIndex])
```

4. **Doet de code wat het moet doen? En is de code duidelijk in wat het doet? In hoeverre komt het overeen met hoe jij het hebt opgelost?**

De code werkt alleen wanneer het bestand wat je wilt openen in de 'root' van je context zit. Een oplossing hiervoor is om het pad van het Python bestand te combineren met de naam van het bestand.

'with open(file_name, 'r') as file:' => 'with open(os.path.join(sys.path[0], file_name), 'r') as file:'



5. **Hoe zien de if/while statements eruit? (Denk aan: duidelijk/clean, zijn er overbodige statements?) In hoeverre komt het overeen met hoe jij het hebt opgelost?**

Er zijn weinig if/while statements gebruikt, maar degene die zijn gebruikt waren nodig. Ook waren ze compact en duidelijk.

6. **Zijn de PEP8 guidelines voor Python toegepast? Waar gaat dat goed? Waar kan het beter?**

De guidelines van PEP8 zijn toegepast.

Dit is te zien aan namen van variabelen, functies en netheid van de code.

7. **Kan de oplossing simpeler? Vergelijk de code van je medestudent met je eigen code van deze opdracht. Wat valt je op?**

Wanneer een bestand geopend wordt is de standaard mode 'r', dit hoeft dus niet nog een keer expliciet meegegeven te worden aan de open functie

```
with open(file_name, 'r') as file:
```

8. **Wordt eventuele verkeerde input afgevangen?**

Verkeerde input wordt goed afgevangen.

Er wordt gekeken of ik een argument heb meegegeven bij het starten van het script.

```
if len(sys.argv) != 2:  
    print("Usage: python3 tail.py <filename>")
```

Bij het openen van het bestand kan er een error ontstaan indien het script een bestand niet kan openen of vinden. Om de functie zit een exception handler, maar die mist "PermissionError".

Ook is het beter dat er eerst gevalideerd wordt of het bestand bestaat en geopend mag worden, voordat het bestand echt geopend wordt met gebruik van 'os.path.isfile()' en 'os.access()'

A. Evaluatie leerdoelen

Professionele Skills

Benoem hier nog jouw beide leerdoelen voor professional skills (het gehele SMART leerdoel):

- Leerdoel week 6: In arch 2 wil ik tijdens elke les maandag aantekeningen maken om mijn concentratie erbij te houden.
Na de les maak ik er een samenvatting van en deel ik deze naar mijn leerteam.
- Leerdoel week 11: Ik wil in arch 3 oefenen met het vinden van betrouwbare technische bronnen. Ik ga elke vrijdag één uur besteden aan het zoeken van bronnen op basis van de op maandag behandelde lesstof.

We willen graag een goed beeld krijgen van je ontwikkeling met betrekking tot *professional skills*. Beantwoord de onderstaande vragen. Zorg voor goede bewijzen. Geef minimaal 1, maximaal 2 bewijzen die jouw ontwikkeling ondersteunen. 

1. **Als je kijkt naar je leerdoelen met betrekking tot professional skills, hoe heb je je dan ontwikkeld op deze leerdoelen? Welke resultaten heb je bereikt? Wat is gelukt en wat niet (helemaal)? Wat is er merkbaar/zichtbaar verbeterd en/of veranderd voor jezelf en anderen?**

Ik heb het gevoel dat door aantekeningen te maken tijdens de lessen, ik de lesstof beter kan onthouden en dat daarnaast ook mijn concentratie een stuk verbeterd is.

De samenvattingen die ik heb gemaakt zijn ook nuttig voor mijn leerteam, omdat sommigen soms wat meer moeite hebben met programmeren en een samenvatting een goede ondersteuning kan zijn.

Doordat ik samenvattingen heb gemaakt kan ik bij onduidelijkheid zelf ook makkelijker terugzoeken in de lesstof.

Ik heb wel gemerkt dat het niet altijd lukt om op maandag na de les de samenvatting te maken, maar hoe langer ik ermee wacht, hoe lastiger het is om alles nog volledig terug te halen, waardoor de samenvatting waarschijnlijk minder uitgebreid is. Het is dus belangrijk om de samenvatting zo snel mogelijk te maken, wanneer de stof nog vers in het geheugen zit.

Het onderzoeken hoe ik het beste technische bronnen kan vinden vond ik wat moeilijker. Ik heb de lesstof van een van de lessen genomen en ben hier verschillende zoektermen voor gaan proberen om te kijken welke de beste resultaten gaven.

Hierbij heb ik gelet op:

- Betrouwbaarheid van de schrijver
- Actualiteit van de bron
- Of de inhoud nuttig is voor mij

Door het oefenen hiermee is het makkelijker geworden om goede bronnen te herkennen voordat ik de inhoud heb gezien.

2. In hoeverre was je aanpak effectief? Wat had je (met je huidige inzicht) kunnen verbeteren in je aanpak?

Door het maken van aantekeningen en samenvattingen is de lesstof beter blijven hangen en kan ik onderdelen makkelijker terugzoeken.

Een verbeterpunt in de aanpak is vooral de samenvatting zo snel mogelijk te maken.

Ook moet ik een manier vinden aantekeningen en samenvattingen op een overzichtelijke manier op te slaan, zodat deze ook makkelijk te vinden zijn tijdens de rest van de opleiding. 

Wat betreft de bronnen is het gelukt hier een effectieve manier voor te vinden.

Op Real Python staan veel bronnen en tutorials die handig zijn om te gebruiken, omdat daar ook praktische voorbeelden in staan.

Deze bronnen en tutorials staan duidelijk gecategoriseerd waardoor informatie snel te vinden is. 

3. Wat neem je mee van wat je geleerd hebt naar de toekomst? (bijv. naar het 2e semester?)

Ik blijf aantekeningen maken tijdens de les en maak de samenvatting er zo snel mogelijk na.

Ik ga op zoek naar een soortgelijke website van Real Python maar dan voor C#, omdat één website waarin heel veel informatie georganiseerd staat handig en snel is.

Programmeren

Benoem hier nog jouw beide leerdoelen voor programmeren (het gehele SMART leerdoel):

- Leerdoel week 6: In arch 2 wil ik elke vrijdag een oude opdracht herzien om nieuw geleerde technieken toe te passen.
- Leerdoel week 11: Ik wil in arch 3 vaker unit tests schrijven. Elke vrijdag ga ik één uur besteden aan unit tests schrijven voor één probleem van de die week behandelde lesstof.

We willen graag een goed beeld krijgen van je ontwikkeling met betrekking tot *programmeren*.

Beantwoord de onderstaande vragen. Zorg voor goede bewijzen. Geef minimaal 1, maximaal 2 bewijzen die je ontwikkeling ondersteunen. 

1. Als je kijkt naar je leerdoelen met betrekking tot programmeren, hoe heb je je dan ontwikkeld op deze leerdoelen? Welke resultaten heb je bereikt? Wat is gelukt en wat niet (helemaal)? Wat is er merkbaar/zichtbaar verbeterd en/of veranderd voor jezelf en anderen?

Door nieuw geleerde technieken toe te passen op oude opdrachten heb ik geleerd dat ik niet te ingewikkeld moet denken en de code niet te uitgebreid moet maken.

Ik heb in veel oude codes de code simpeler kunnen maken door list/dict comprehension toe te passen. 

Ook heeft dit leerdoel mij laten realiseren dat het soms handiger is om sommige functies volledig opnieuw te schrijven wanneer de opdracht te veel veranderd is, omdat het aanpassen

van de oude code minder efficiënt, leesbaar en netjes zou zijn dan wanneer ik het opnieuw zou schrijven. Ook duurt het aanpassen soms langer dan het opnieuw schrijven.

Het resultaat wat ik heb bereikt is dat ik unit tests heb bedacht en geschreven.

Het schrijven van de unit tests lukte goed voor weken 9 en 11, alleen is het bij de opdrachten van week 10 niet gelukt, omdat deze niet echt units hadden die getest konden worden. Toen ik aan Ditmar vroeg hoe dit moest, zei hij dat het geen unit tests waren maar integration tests. Ik wist in grote lijnen wat een unit test was, maar niet in de diepte wat het allemaal precies in hield. Daarom had ik twee opdrachten van week 9 gedaan.

2. In hoeverre was je aanpak effectief? Wat had je (met je huidige inzicht) kunnen verbeteren in je aanpak?

Door het toepassen van de nieuw geleerde technieken bij bestaande opdrachten heb ik deze kunnen verbeteren en daardoor was de aanpak effectief.

Wat betreft de unit tests maken, was de aanpak effectief. Ik zou wel nog integration tests willen doen, waarbij de output in de console ook gecheckt wordt. Zo hadden de opdrachten van week 10 getest kunnen worden, wat met unit tests niet kon.

3. Wat neem je mee van wat je geleerd hebt naar de toekomst? (bijv. naar het 2e semester?)

Ik heb geleerd dat ik codes soms minder ingewikkeld en uitgebreid kan maken. Ook zal ik eerder sommige functies volledig opnieuw schrijven dan aanpassen, omdat het aanpassen van een oude code soms de kwaliteit verlaagt en vaak ook meer tijd kost.

Ik zie nu meer het nut van unit tests in en zal ze ook vaker in mijn eigen projecten gaan gebruiken. Op mijn werk zal ik ook aanraden om dit te gebruiken, omdat het handig is als vangnet op de tester om zo fouten te ontdekken voordat het op de productie omgeving staat.

B. Evaluatie leeruitkomsten

Professionele Skills

Student succes

- ***Wat kan ik al goed?***

Ik vind dat het plannen van de technische opdrachten goed gaat.

De programmeeropdrachten probeer ik altijd op de dag zelf te doen, dit lukt 90% van de tijd. 

Als dat niet lukt doe ik dat op de vrijdag.

Mijn motivatie ligt bij het programmeren, maar niet bij schrijfwerk, onderzoeken of andere niet-programmeer gerelateerde opdrachten.

- ***Wat kan ik nog verbeteren?***

De planning van opdrachten waarin onderzoek of veel schrijfwerk in voor komt stel ik vaak tot de laatste week uit, dit zorgt ervoor dat ik in die laatste week er soms dagen tot middernacht aan zit.

Ook ben ik in het algemeen niet goed in schrijfwerk, een oorzaak hiervan is omdat ik begin met typen voordat concreet heb wat ik moet schrijven, daardoor verandert er veel en moet ik het meestal opnieuw schrijven.

Ondanks dit haal ik toch wel elke deadline.

Research

- ***Wat kan ik al goed?***

Na het leerdoel dat ik in arch 3 heb gedaan, heb ik de impressie dat het zoeken van betrouwbare bronnen en het vermelden ervan mij nu makkelijker afgaat.

Ik heb ermee geoefend door verschillende zoektermen te testen om te zien welke de beste resultaten oplevert.

In het kader van initiatief nemen bel ik elke woensdag met mijn leerteam voor vragen en als check-up om te kijken of het bij iedereen lukt.

- ***Wat kan ik nog verbeteren?***

Hoewel bronvermeldingen met gebruik van IEEE in Word ingevulden zitten weet ik niet precies wat de exacte regels zijn, hierdoor moet ik vertrouwen op dat Word het goed doet. Wat eigenlijk niet gewenst is.

Problem Solving

- ***Wat kan ik al goed?***

Error berichten bij het runnen van de code kan ik goed omzetten naar een stappenplan om de error op te lossen. Ook ben ik goed in het specifiek zoeken op Google naar een oplossing voor de error. 

- ***Wat kan ik nog verbeteren?***

Soms kan ik beter vooruitdenken zodat eventuele problemen of errors voorkomen zouden kunnen worden. Dit houdt in dat ik meer tijd zou kunnen besteden aan het plannen voordat ik aan de slag ga met het coderen.

Programmeren

Basic and advanced structures

- ***Wat kan ik al goed?***

Ik herken structuren goed en kan deze dan ook maken  zonder problemen. Dit komt door mijn vorige opleiding.

Deze kennis ga ik ook toe moeten passen in andere talen zoals bijvoorbeeld Typescript en C# waardoor deze kennis heel fijn is om te hebben.

- ***Wat kan ik nog verbeteren?***

Bij het uitwerken zie ik soms nog dingen over het hoofd waardoor ik het achteraf moet aanpassen. Ook gebruik ik uit automatisme vanuit eerdere kennis voornamelijk lists terwijl er in Python ook tuples bestaan die ik kan gebruiken.

Debugging and testing

- ***Wat kan ik al goed?***

Bij het debuggen kan ik redelijk makkelijk spotten waar de fout zit, dit maakt de tijd dat het kost om op te lossen veel korter. Qua testen doe ik bij elk nieuw stukje functionaliteit testen of het werkt voordat ik verder ga. Dit voorkomt dat als één stuk niet werkt, ik niet de halve code hoeft te herschrijven.

Ook het schrijven van unit tests doe ik consistent, waardoor er nog een extra laag aan tests op de geschreven code zit.

- ***Wat kan ik nog verbeteren?***

Ik wil de code vaak te uitgebreid maken om alle scenario's af te vangen, dit kost eigenlijk te veel tijd, maakt functies veel te ingewikkeld en onduidelijk, en is ook niet altijd nodig.

Dataprocessing

- ***Wat kan ik al goed?***

Het opslaan van data in CSV, JSON en met gebruik van een database gaat gemakkelijk, dit is omdat ik hier al eerder mee heb gewerkt.

Ook heb ik al ervaring met uitgebreidere SQL-statements zoals inner joins en het gebruik van functies binnen SQL.

- ***Wat kan ik nog verbeteren?***

Het verwerken van data dat geen tekst is zoals afbeeldingen, video en audio ben ik nog niet bekend mee, hier zou ik mij wel in willen verdiepen omdat ik het een interessant onderwerp vindt.

Ook ben ik nog niet enorm bekend met het verwerken van grote datasets zodat het zo efficiënt mogelijk gaat.

C. Evaluatie Basecamp

Je hebt nu een evaluatie geschreven over je leerdoelen en de leeruitkomsten. Wij zijn ook benieuwd hoe je Basecamp hebt ervaren; waar ben je begonnen en waar sta je nu?

1. Beschrijf hier hoe je terugkijkt op Basecamp. Hoe heb je het ervaren? Wat heb je gaandeweg geleerd?

Ik heb Basecamp ervaren als een hele leerzame periode.

Door mijn vooropleiding had ik voordat ik met de opleiding begon al een beetje ervaring met programmeren in Python, maar tijdens BaseCamp heb ik veel nieuwe dingen geleerd vooral over de theorie erachter zoals bijvoorbeeld het verschil tussen mutable en immutable types.

Voordat ik met de opleiding begon vond ik Python niet echt een fijne taal om mee te werken, maar dit is nu compleet anders. Ik gebruik het nu ook voor kleine taken om mezelf het leven makkelijker te maken, bijvoorbeeld door een script te schrijven die automatisch foto's sorteert en verplaatst na het fotograferen.

Ook heb ik naast het technische aspect mijzelf ontwikkeld in professional skills zoals het zoeken van technische bronnen, het uitwerken van samenvattingen en hetgeen waar ik me het meeste in heb ontwikkeld is het uitschrijven van opdrachten. Dit was het gedeelte waar ik het meeste tegenop zag toen ik met de opleiding wilde starten. Ik vind dit nog steeds lastig maar merk dat ik er wel beter in word.

Ik vind het prettig hoe hecht mijn leerteam is, het is fijn dat we elkaar kunnen ondersteunen op gebieden waar de ander minder goed in is, bijvoorbeeld Gülseren is goed in het kritisch naar teksten kijken en helpt mij hier soms mee, terwijl zij haar vragen met betrekking tot programmeren aan mij stelt.

2. Waar ben je het meest trots op (zowel op persoonlijk vlak als op technisch vlak)?

Persoonlijk:

Ik ben er trots op dat ik in het eerste half jaar op het hbo naar mijn mening goed heb gepresteerd.

Ik kan goed meekomen met het tempo van de lesstof, ik heb mijn plek gevonden in het leerteam waar ik een goede bijdrage in kan leveren, en misschien wel het belangrijkste: ik ga met plezier naar school. Dit was bij mijn vorige opleiding niet altijd het geval.

Technisch:

Ik ben trots op dat alle assignments en problems die ik gemaakt heb afgerond met een 10 omdat dit aantoont dat ik de technieken die hiervoor nodig zijn goed onder de knie heb.

Ook ben ik trots op de game die ik heb gemaakt in de challenge weeks, hier heb ik technieken moeten gebruiken die ik nog niet eerder gebruikt heb zoals udp en https in Python, daarom was dit naast een leuke game ook een goed leermoment.

3. Geef een tip en een top met betrekking tot Basecamp.

Tip:

Sommige opdrachten zijn een beetje onduidelijk en soms kloppen de comments in de voor aangeleverde code niet met het doel van de opdracht.

Ook zitten er redelijk veel taalfouten in de formats en opdrachten die wij aangeleverd krijgen.

Top:

Het is fijn dat er elke week een duidelijke klassikale uitleg wordt gegeven, je wordt dus niet in het diepe gegooid met zelfstudie

Ook vind ik het prettig dat de opdrachten in CodeGrade ingeleverd moeten worden waardoor ze meteen nagekeken worden en je direct feedback hebt op de werking van de code maar ook op de manier hoe de code is geschreven, bijvoorbeeld op het naleven van code standards.

2. Ondersteunend bewijs

2.1 Supporting topic week 2 – Data formats

Handmatig omrekenen

1. **Cijfers in decimale getallen zijn 0-9. Wat zijn de cijfers in hexadecimaal formaat? Wat zijn de cijfers in binair formaat?**

Decimaal	0	1	2	3	4	5	6	7	8	9
Binair	0	1	10	11	100	101	110	111	1000	1001
Hexadecimaal	0	1	2	3	4	5	6	7	8	9

2. **Converteer (handmatig) de volgende decimale getallen naar hexadecimaal en binair: 8, 10, 15, 21, 32, 64, 256, 500, 512, 1000.**

Decimaal	8	10	15	21	32	64	256	500	512	1000
Binair	1000	1010	1111	10101	100000	1000000	100000000	111110100	1000000000	1111101000
Hexadecimaal	8	A	F	15	20	40	100	1F4	200	3F8

3. **Hoe geeft Python deze data formats weer? Hoe kun je Python gebruiken om deze data forms naar elkaar te converteren? Beschrijf hieronder in je eigen woorden.**

In Python kan je binary en hex naar int converteren door `int()` te gebruiken met als tweede argument de "base". Dat is voor binair 2 en bij hexadecimaal 16.

Om van int naar binary te gaan kan je de `bin()` functie gebruiken en om van int naar hex te gaan kan je de `hex()` functie gebruiken.

Binary wordt als `0b<binary waarde>` weergegeven.

Hex wordt als `0x<hex waarde>` weergegeven.

4. **Ga op onderzoek uit en geef hieronder een voorbeeld uit de praktijk waarbij binaire gegevens veelvuldig worden gebruikt.**

Binair is de basis van alle moderne technologie, een voorbeeld hiervan is een CPU, die werkt met binaire gegevens om instructies te verwerken [1].

5. **Onderzoek hoe hexadecimaal wordt gebruikt bij het aanspreken van het computergeheugen. Leg het kort in je eigen woorden uit.**

Hexadecimale worden gebruikt om het adres van computergeheugen te noteren.

De reden waarom hier geen binair voor gebruikt wordt is omdat hexadecimale korter en minder foutgevoelig zijn [2].

Onderzoek hoe decimale gegevensformaten worden gebruikt in financiële berekeningen of boekhoudsystemen. Leg het kort in je eigen woorden uit.

Het decimale format wordt gebruikt bij financiële berekeningen, omdat daar veel met komma getallen gewerkt kan worden.

Decimals zijn daar nauwkeurig in, waar floats en doubles dat minder zijn [3].

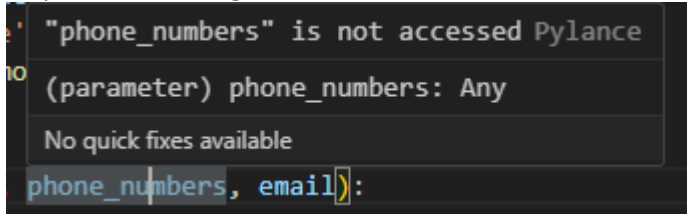
6. Van welke bronnen heb je gebruik gemaakt bij het doen van onderzoek (websites, boeken, artikelen). Geef hier de door jouw gebruikte bronnen weer.

- Harvard, "Lecture 4," 2021. [Online]. Available: - <https://cs50.harvard.edu/college/2021/spring/notes/4/>.
- D. Hua, A. Weiland and S. Drummond, Bits, Bytes, and Binary: The Mathematics of Computers, Tech Directions, 2014.
- F. Mudawar, Exact Versus Inexact Decimal Floating-Point Numbers and Arithmetic, IEEE Access, vol. 11,, 2023.



2.2 Supporting topic week 6 – Debugging

Bevinding #1	Script kan niet geïmporteerd worden als module.
a. Wat voor probleem veroorzaakt de bug?	Er is geen check gedaan of het script wordt geïmporteerd of niet
b. Waar in de code heb je de bug gevonden (regelnummer)?	Regel 61
c. Wat heb je gedaan om de bug te vinden?	Ik heb geprobeerd om het script te importen als module via een ander script
d. Hoe heb je bug opgelost?	Een <code>"if __name__ == '__main__':"</code> statement om de <code>main()</code> functie gezet

Bevinding #2	Telefoonnummers worden niet opgeslagen bij het aanpassen van contact
a. Wat voor probleem veroorzaakt de bug?	De parameter <code>"phone_numbers"</code> wordt niet gebruikt in de functie
b. Waar in de code heb je de bug gevonden (regelnummer)?	23
c. Wat heb je gedaan om de bug te vinden?	Ik heb door de code heen gescand en de IDE duidde aan dat de parameter niet gebruikt werd 
d. Hoe heb je bug opgelost?	Op regel 26 <code>"contact['phone_numbers'] = name"</code> aangepast naar <code>"contact['phone_numbers'] = phone_numbers"</code>

Bevinding #3	Telefoonnummers worden incorrect opgeslagen
a. Wat voor probleem veroorzaakt de bug?	De input prompt vraagt <code>"Enter the new phone numbers (separated by commas):"</code> waar vervolgens de input op een <code>"."</code> Gesplit wordt
b. Waar in de code heb je de bug gevonden (regelnummer)?	55
c. Wat heb je gedaan om de bug te vinden?	Ik heb een contact aangepast en daarna opnieuw opgezocht en merkte dat alle nummers als één string opgeslagen werden en niet als meerdere
d. Hoe heb je bug opgelost?	De functie waar er op een <code>"."</code> gesplit wordt, aangepast naar een <code>","</code>

Bevinding #4	Zoeken wordt op email gedaan in plaats van op naam
a. Wat voor probleem veroorzaakt de bug?	In de lambda functie wordt er naar "c['email']" gekeken in plaats van "c['name']"
b. Waar in de code heb je de bug gevonden (regelnummer)?	14
c. Wat heb je gedaan om de bug te vinden?	Tijdens het zoeken vulde ik "John Doe" in en kreeg ik geen resultaten. Toen heb ik in de search de lijst geprint en merkte dat het contact wel in de lijst stond. Na het kijken naar de "search_contacts" zag ik de bug.
d. Hoe heb je bug opgelost?	"c['email']" omgezet naar "c['name']"

1. Beschrijf hier welke tips en bronnen je andere studenten kan geven over het leren debuggen.

Het volgende zou ik andere studenten aan willen raden over het efficient leren debuggen.

- Kijk goed naar eventuele aanwijzingen in je IDE (bijvoorbeeld highlighting). Syntax errors worden al met een rood lijntje onder de plek waar het ontstaat aangegeven.
- Lees de vragen goed en bekijk de functies om te zien of ze ook echt uitvoeren wat er gevraagd wordt.
- In de onderstaande bron zijn er nog meer handige tips te vinden, ook komen daar veelvoorkomende foutmeldingen in terug.

<https://www.freecodecamp.org/news/python-debugging-handbook>

2.3 Supporting topic week 7 – Unit testen

- 1 Zoek uit wat een unit test is. Gebruik hierbij ten minste drie verschillende bronnen, zoals websites, video's of documentatie. Bantwoord hierbij de volgende vragen:

- **Leg in je eigen woorden uit wat een unit test is.**
Een unit test is een automatische test die geschreven kan worden om een specifieke functie te testen [4].
- **Waarom worden unit tests gebruikt in softwareontwikkeling?**
Om automatisch en moeiteloos functies te testen, de test wordt één keer geschreven en kan daarna altijd opnieuw gebruikt worden.
- **Wat voor testsoorten zijn er nog meer? Noem er minimaal 2 en leg de verschillen met een unit test uit.**

Er zijn nog meerdere testsoorten, "User Acceptance Testing (UAT)" en "Integration Testing".

User Acceptance Testing houdt in dat de eindgebruiker naar het product kijkt en test, uit mijn ervaring komt dit meestal in een vorm van een beta versie van de software. Dit gebeurt dus handmatig, tegenovergesteld van unit testen wat automatisch is [5].

Integration testing is een uitbreiding op unit testing doordat in die test methode units bij elkaar getest worden om te zien of zij goed met elkaar werken [6].

- **Wat zijn assert statements? Bekijk een aantal voorbeelden; deze helpen je bij het uitvoeren van opdracht 2.**
Een assert statement kijkt of een expression "True" of "False" is, op het moment dat het true is gebeurt er niks, maar als de expression "False" returnt dan wordt er een "AssertionError" gegeven.
- **Noem een paar voordelen en nadelen van unit testing.**
Het maken van unit tests heeft als voordeel dat de tests maar één keer geschreven hoeven te worden en daarna continue gebruikt kunnen worden.

Tijdens het programmeren heb je soms dat als je één functie aanpast, dat een andere stopt met werken. Wanneer geschreven vangen unit tests dit af.

Het nadeel is dat het extra tijd kost boven op de al nodige development tijd.
Als het product veel verandert is het soms niet de moeite waard om een unit test te schrijven.

2.4 Supporting topic week 10 – Exception handling

A. Onderzoek naar exception handling

1. Van welke medestudent heb jij vragen ontvangen?
Naam: Gülseren Kasaroğlu
2. Welke 3 vragen heb jij gekregen? En wat is het antwoord dat je gevonden hebt op basis van jouw onderzoek?
 1. Vraag: Wat is het doel van exception handling?
Antwoord: Het doel van exception handling is om te voorkomen dat runtime errors (errors die voorkomen tijdens het uitvoeren van de code) ervoor zorgen dat het programma crasht [7].
 2. Vraag: Wat zijn de verschillen in exception handling tussen Python en andere programmeertalen?
Antwoord: Er zijn geen fundamentele verschillen in error handling tussen Python en andere programmeertalen, wel wordt het anders geschreven.
Bijvoorbeeld in c# wordt er "catch" gebruikt in plaats van "except" en word "raise" vervangen door "throw" [7] [8].
 3. Vraag: Wanneer gebruik je exception handling?
Antwoord: De beste werkwijze is om overal exception handling toe te voegen waar er een error kan voorkomen [9].
3. Welke bronnen heb je gebruikt voor je onderzoek. Benoem er minimaal 3.
 - S. Klundert, "Python Exceptions: An Introduction," 29 1 2024. [Online]. Available: <https://realpython.com/python-exceptions/>.
 - B. Lubanovic, Introducing Python: Modern Computing in Simple Packages, O'Reilly Media, 2019.
 - Microsoft, "Exception-handling statements," 22 4 2023. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements>.

B. Toepassen van exception handling

Inleiding: In Python zijn 'exceptions' onverwachte gebeurtenissen die kunnen voorkomen tijdens het runnen van een programma en die de normale flow van code kunnen verstoren. Dit kunnen fouten zijn, zoals het proberen om een niet-bestaand bestand te openen.

Om deze fouten op te vangen en te voorkomen dat het programma crasht, heeft Python een hulpmiddel; dit noemen we een try-except-block. De code die een exception zou kunnen oproepen, wordt in het try block geplaatst. Als er een exception optreedt binnen het try-block, gaat Python direct naar het bijpassende except-block, waar de fout kan worden opgevangen en afgehandeld. Dit helpt ontwikkelaars om mogelijke fouten af te vangen en de juiste oplossing te bedenken, zodat het programma goed blijft werken, zelfs als er iets onverwachts gebeurt. Door try-except blocks te gebruiken, worden Python-programma's robuuster, omdat fouten kunnen worden afgehandeld zonder dat het programma plotseling stopt.

Nu gaan we het toepassen:

- Maak een eenvoudig voorbeeld met een opzettelijke fout, zoals het proberen te openen van een bestand met een verkeerde naam. Voer het programma uit en bekijk het resultaat.
- Verbeter je programma door een try-except block toe te voegen.
- Raised exceptions zijn objecten. Ze hebben methodes om te kunnen zien van welk type ze zijn en error berichten weer te geven. Gebruik deze objecten om de juiste foutmelding weer te geven aan de gebruiker wanneer er een uitzondering optreedt.
- Verbeter de code van de [Code Analysis opdracht](#) met exception handling.
- Voeg de verbeterde code toe aan de bijlagen van je dossier.

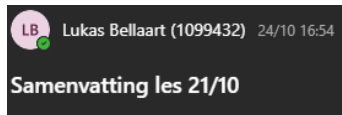
Bronnenlijst

- [1] D. Hua, A. Weiland and S. Drummond, Bits, Bytes, and Binary: The Mathematics of Computers, Tech Directions, 2014.
- [2] Harvard, "Lecture 4," 2021. [Online]. Available: -
<https://cs50.harvard.edu/college/2021/spring/notes/4/>.
- [3] . F. Mudawar, Exact Versus Inexact Decimal Floating-Point Numbers and Arithmetic, IEEE Access, vol. 11,, 2023.
- [4] AWS, "Amazon AWS," [Online]. Available: <https://aws.amazon.com/what-is/unit-testing/>.
- [5] Stanford University, "User Acceptance Testing," [Online]. Available:
<https://uit.stanford.edu/pmo/UAT>.
- [6] Stanford University, "System Integration Testing," [Online]. Available:
<https://uit.stanford.edu/pmo/SIT>.
- [7] S. Klundert, "Python Exceptions: An Introduction," 29 1 2024. [Online]. Available:
<https://realpython.com/python-exceptions/>.
- [8] B. Lubanovic, Introducing Python: Modern Computing in Simple Packages, O'Reilly Media, 2019.
- [9] Microsoft, "Exception-handling statements," 22 4 2023. [Online]. Available:
<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements>.

3. Bijlagen

3.1 Overige noodzakelijke bijlagen, die bijdragen aan het bewijzen van de leeruitkomsten Bijlage 1.3

Professionele leerdoel:



Programmeren:

Lambda:

- Is een anonymous function
- Kan je gebruiken wanneer een functie niet vaker dan 1 keer gebruikt wordt
- Meestal gebruikt op filters, sorts en maps.
- "lambda x: <equation>

If `__name__ == "__main__":`

- Word gebruikt om te voorkomen dat alle code (eventueel ongewenste) ge-execute wordt bij het laden van het bestand.

- Door te checken of `__name__ == "__main__"` is, voorkom je dat
- `__main__` is het hoofdbestand dat gedraaid wordt
- altijd het laatste in je document

List comprehension:

- `[age for age in ages if age >= 18]` Filtert ages op aantallen die groter zijn dan 18
- Is sneller dan een uitgeschreven for loop aangezien minder pointers.

Dict comprehension:

- `{key: value for key, value in student.items() if key != "id"}` creert een nieuwe dict maar dan zonder id.

Nested data structures:

- Items apart van elkaar bekijken

Lotte dr gedeelte:

Doelen moeten vanaf vandaag gedaan worden (21/10)

Plan van Aanpak:

Leerdoelen staan in "Basecap cursushandleiding 2024-2025.pdf"

Er moeten twee leerdoelen gekozen worden per arch.

Peer evaluatie:

Verzamel:

- Feedback op handelen

Supporting topics Unit testing:

Format moet ingevuld worden

Er moeten 3 bronnen gebruikt worden.

TUSSENTIJDSE INLEVERDATUM DOSSIER: 8 november

LB

Lukas Bellaart (1099432) 16:10

Samenvatting les 4/21

Vandaag was er weinig les vanwege de challenge week.

Deze week is het de bedoeling dat je de zelfevaluatie maakt.
Deze maandag voortgang gesprekken met Lotte en Ditmar.

Vrijdag 18:00 moet het tussentijdse dossier t/m week 8 worden ingeleverd.
Probeer hierbij ook alvast bronvermelding zodat hierop feedback gegeven kan worden.

Technisch leerdoel:

Aanpassingen op [A1W3A1 - Predefined templates](#)

- Alle validatiefuncties zijn verkort en versimpeld zodat de code compacter en overzichtelijker is.

Eerst:

```
def is_name_valid(name: str) -> bool:
    if not 2 <= len(name) <= 10:
        return False
    if not name.isalpha():
        return False
    if not name[0].isupper():
        return False
    return True

def is_title_valid(title: str) -> bool:
    if len(title) < 10:
        return False
    if any(character.isdigit() for character in title):
        return False
    return True

def is_salary_valid(salary: str) -> bool:
    return MIN_SALARY <= float(salary.replace(".", "").replace(",", ".")) <= MAX_SALARY
```

Na:

```
def is_name_valid(name: str) -> bool:
    return 2 <= len(name) <= 10 and name.isalpha() and name[0].isupper()

def is_title_valid(title: str) -> bool:
    return len(title) >= 10 and not any(character.isdigit() for character in title)
```

```
def is_salary_valid(salary: str) -> bool:
    return MIN_SALARY <= float(salary.replace(".", "").replace(",",
".")) <= MAX_SALARY
```

- De constante helemaal bovenaan in de code gezet in plaats van in het midden, zodat deze makkelijker aan te passen is.
- In de "get_input" functie de parameter genaamt "type" vervangen met "prompt" aangezien type een gereserveerde naam is.
- In de main loop heb ik een guard clause toegepast indien "continue_input" "nee" is. Dit voorkomt dat de if statement dieper genest wordt, dat maakt de code leesbaarder

Aanpassingen op [A2W5A1 - Processing student data](#)

- In de "is_valid_date" functie heb ik een map toegepast om de gesplitste datum om te zetten naar het juiste type, zodat later niet apart hoeft.

Voor:

```
def is_valid_date(date: str) -> bool:
    if not len(date.split("-")) == 3:
        return False

    year, month, day = date.split("-")

    if not 1960 <= int(year) <= 2004:
        return False

    if not 1 <= int(month) <= 12:
        print(month)
        return False

    if not 1 <= int(day) <= 31:
        print(day)
        return False

    return True
```

Na:

```
def is_valid_date(date: str) -> bool:
    year, month, day = map(int, date.split("-"))
    return (1960 <= year <= 2004) and (1 <= month <= 12) and (1 <= day <= 31)
```

- Daarna heb ik in dezelfde functie meerdere if statements samengevoegd tot één statement, waardoor de code compacter is.
- Overall waar in list aan gemaakt werd met gebruik van "[]", heb ik list() van gemaakt omdat dat een duidelijkere manier is om een lege lijst aan te maken
- Op functie "validate_data" heb ik type hinting toegevoegd zodat de verwachte types duidelijk zijn.
- De if statement die checkt of dat het studentnummer met "08" of "09" begint is nu versimpeld.
- De lijst die "student_program" valideert heb ik omgezet in een set, omdat dat sneller is

Professionele leerdoel:

Betrouwbare bron voor [week 9: Everything is an Object.](#)

https://opus.us.edu.pl/fileView.seam?fileName=Hernandez-Figueroa_Python_Classes.pdf&affil=&entityType=book&fileTitle=Hernandez-Figueroa_Python_Classes.pdf+%2F+1+MB+%2F+&entityId=USL03255d049e514682a2e038bf839bb659&lang=pl&fileId=USL3f5da3ee4c49433c87ff48a409ac18d6&cid=473346

Voor deze bron heb ik in Google Scholar gezocht naar “Python” en “Classes”, dit was één van de eerste resultaten. Dit is een fijne bron omdat ze voorbeelden met gebruik van code geven maar ook praktische voorbeelden.

De stof die behandeld wordt gaat beginner naar heel geavanceerd.

De stof lijkt betrouwbaar en relevant te zijn, dit concludeer ik uit meerdere redenen:

- Het artikel is gepubliceerd in 2021
- Het is gepubliceerd door een universiteit
- Meerdere auteurs van het artikel zijn professors bij universiteiten.

Betrouwbare bron voor [week 10: \(Plain\) Data Files](#)

<https://www.dataquest.io/blog/read-file-python/>

Ik heb in Google Scholar zoektermen gebruikt als: "Python" en "Text", “Reading text with python” en “Opening text files with python”

Bij de eerste twee zoektermen ging het onderwerp van de bronnen voornamelijk over het analyseren van tekst en niet over het openen en verwerken ervan. Bij de laatste zoekterm kreeg ik een bron die in principe genoeg informatie gaf, maar ik vond het net niet genoeg dus besloot ik buiten Google Scholar te zoeken.



Met de zoekterm “reading text file in python” vond ik deze bron.

Deze bron is recent, zeer uitgebreid en legt ook meerdere eventuele foutmeldingen uit. Ook zit er bij deze bron veel tutorials voor brede en specifieke gebruiksgevallen.

Deze bron oogt betrouwbaar omdat de auteur bekend is en er een stuk over wat zijn rol/ervaring is.


```
if __name__ == "__main__":  
    main()
```

Unit tests voor [A3W09P4 - Distance Converter](#)

```
from distanceconverter import Converter  
from unittest import TestCase, main  
  
class TestDistanceConverter(TestCase):  
    converter = Converter(9, "inches")  
  
    def test_inches(self):  
        self.assertEqual(self.converter.inches(), 9)  
  
    def test_feet(self):  
        self.assertEqual(self.converter.feet(), 0.75)  
  
    def test_yards(self):  
        self.assertEqual(self.converter.yards(), 0.25)  
  
    def test_miles(self):  
        self.assertEqual(round(self.converter.miles(), 9), 0.000142045)  
  
    def test_kilometers(self):  
        self.assertEqual(self.converter.kilometers(), 0.0002286)  
  
    def test_meters(self):  
        self.assertEqual(self.converter.meters(), 0.2286)  
  
    def test_centimeters(self):  
        self.assertEqual(self.converter.centimeters(), 22.86)  
  
    def test_millimeters(self):  
        self.assertEqual(self.converter.millimeters(), 228.6)  
  
    def test_default_feet(self):  
        converter = Converter(9, "feet")  
        self.assertEqual(round(converter.meters(), 4), 2.7432)  
  
    def test_default_yards(self):  
        converter = Converter(9, "yards")  
        self.assertEqual(converter.meters(), 8.2296)  
  
    def test_default_miles(self):  
        converter = Converter(9, "miles")  
        self.assertEqual(round(converter.meters(), 3), 14484.096)  
  
    def test_default_kilometers(self):
```

```

        converter = Converter(9, "kilometers")
        self.assertEqual(converter.meters(), 9000)

    def test_default_meters(self):
        converter = Converter(9, "meters")
        self.assertEqual(converter.meters(), 9)

    def test_default_centimeters(self):
        converter = Converter(9, "centimeters")
        self.assertEqual(converter.meters(), 0.09)

    def test_default_millimeters(self):
        converter = Converter(9, "millimeters")
        self.assertEqual(round(converter.meters(), 3), 0.009)

    def test_invalid_unit(self):
        with self.assertRaises(ValueError):
            Converter(9, "invalid")

if __name__ == "__main__":
    main()

```

Unit tests voor A3W11P1 - [Movie collection](#)

```

from moviecollection import MovieArchive
from unittest import TestCase, main
import os
import sys

class TestMovieArchive(TestCase):
    with open(os.path.join(sys.path[0], "movies.json"), "r", encoding="utf-8")
    as file:
        with open(os.path.join(sys.path[0], "movies_test.json"), "w",
        encoding="utf-8") as new_file:
            new_file.write(file.read())

    archive = MovieArchive("movies_test.json")

    def test_single_movie_search(self):
        search_result = self.archive.search(title="Ocean's Eleven")
        self.assertEqual(len(search_result), 1)
        self.assertEqual(search_result[0]["title"], "Ocean's Eleven")

    def test_multiple_movie_search(self):
        search_result = self.archive.search(
            title="Ocean's", cast=["Brad Pitt", "George Clooney"]

```

```

    )
    self.assertEqual(len(search_result), 3)

def test_cast_search(self):
    search_result = self.archive.search(cast=["Brad Pitt"])
    self.assertEqual([movie["title"]
                      for movie in search_result].count("Ocean's Eleven"),
1)

def test_edit_title_movie(self):
    self.archive.edit("John Wick: Chapter 2",
                      new_title="John Doe: Chapter 2")
    search_result = self.archive.search(title="John Doe: Chapter 2")
    self.assertEqual(len(search_result), 1)
    self.assertEqual(search_result[0]["title"], "John Doe: Chapter 2")

def test_edit_year_movie(self):
    self.archive.edit("Inglourious Basterds", new_year=2022)
    search_result = self.archive.search(title="Inglourious Basterds")
    self.assertEqual(len(search_result), 1)
    self.assertEqual(search_result[0]["year"], 2022)

def test_edit_cast(self):
    self.archive.edit_cast({"Amber Heard": None})
    search_result = self.archive.search(cast=["Amber Heard"])
    self.assertEqual(len(search_result), 0)

if __name__ == "__main__":
    main()

```

Bijlage 2.1 Supporting topic week 2 – Data formats

<https://app.codegra.de/courses/7735/assignments/107494/submissions/10256207/files/122514114>

Bijlage 2.2 Supporting topic week 6 – Debugging

```
contacts = []

def add_contact(name, phone_numbers, email):
    contact = {
        'name': name,
        'phone_numbers': phone_numbers,
        'email': email
    }
    contacts.append(contact)

def search_contacts(keyword):
    return list(filter(lambda c: keyword.lower() in c['name'].lower(), contacts))

def delete_contact(name):
    for contact in contacts:
        if contact['name'].lower() == name.lower():
            contacts.remove(contact)

def update_contact(name, phone_numbers, email):
    for contact in contacts:
        if contact['name'].lower() == name.lower():
            contact['phone_numbers'] = phone_numbers
            contact['email'] = email
            break

def main():
    add_contact("John Doe", ["1234567890", "9876543210"], "john@example.com")
    add_contact("Jane Smith", ["5555555555"], "jane@example.com")
    add_contact("Bob Johnson", ["1111111111",
                                "2222222222", "3333333333"], "bob@example.com")

    search_term = input("Enter a name to search: ")
    search_results = search_contacts(search_term)

    if search_results:
        print("Search Results:")
        for contact in search_results:
            print(f"Name: {contact['name']}")
            print("Phone Numbers:", ', '.join(contact['phone_numbers']))
            print(f"Email: {contact['email']}")
    else:
        print("No matching contacts found.")

    contact_name = input("Enter the name of the contact to delete: ")
    delete_contact(contact_name)
    print("Contact deleted successfully.")
```



```

update_name = input("Enter the name of the contact to update: ")
update_phone_numbers = input(
    "Enter the new phone numbers (separated by commas): ").split(",")
update_email = input("Enter the new email address: ")
update_contact(update_name, update_phone_numbers, update_email)
print("Contact updated successfully.")

if __name__ == "__main__":
    main()

```

Bijlage 2.3 Supporting topic week 7 – Unit testen

```

from unittest import TestCase, main
from unit_testing import add_contact, search_by_name, delete_contact, contacts

class TestContactMethods(TestCase):

    def setUp(self):
        contacts.clear()

    def test_add_contact(self):
        add_contact('John Doe', '06123456790', 'john.doe@email.com')
        self.assertEqual(len(contacts), 1)
        self.assertEqual(contacts[0].get('name'), 'John Doe')
        self.assertEqual(contacts[0].get('phone_number'), '06123456790')
        self.assertEqual(contacts[0].get('email'), 'john.doe@email.com')

    def test_search_by_name(self):
        add_contact('Jane Doe', '06123456790', 'john.doe@email.com')
        search_results = search_by_name('Jane')
        self.assertEqual(len(search_results), 1)
        self.assertEqual(search_results[0].get('name'), 'Jane Doe')

    def test_delete_contact(self):
        add_contact('John Doe', '06123456790', 'john.doe@email.com')
        delete_contact('John Doe')
        self.assertEqual(len(contacts), 0)
        search_results = search_by_name('John')
        self.assertEqual(len(search_results), 0)

if __name__ == '__main__':
    main()

```

Bijlage 2.4 Supporting topic week 10 – Exception handling

```
class TemperatureDataAnalyzer:
    def __init__(self, file_path):
        self.file_path = file_path
        self.temperature_data = []

    # Method to open the file and load lines as an attribute
    def load_data(self):
        try:
            with open(self.file_path, 'r') as file:
                data = [line.strip().split() for line in file]
                self.temperature_data = [
                    list(map(int, d[:-1]))+[float(d[len(d)-1])] for d in data]
        except FileNotFoundError:
            print('File not found')
        except ValueError:
            print('Invalid data in the file')

    # Method to perform the analysis and construct the list
    def construct_temperature_list(self):
        temperature_list = []
        for data in self.temperature_data:
            try:
                month, day, year, temperature = data[:]
                if year not in [item[0] for item in temperature_list]:
                    temperature_list.append((year, {}))
                if month not in temperature_list[-1][1]:
                    temperature_list[-1][1][month] = 0.0
                temperature_list[-1][1][month] = max(
                    temperature, temperature_list[-1][1][month])
            except ValueError as error:
                print('Invalid data in the file', error)
            except IndexError as error:
                print('Invalid data in temperature data', error)

        return temperature_list

def main():
    # File is build up of year, month, day, temperature, separated by space
    file_path = './temps.txt'
    analyzer = TemperatureDataAnalyzer(file_path)
    analyzer.load_data()
    temperature_list = analyzer.construct_temperature_list()
    # Output: [(2019, {1: 32.0, 2: 45.0, 3: 50.0}), (2020, {1: 35.0, 2: 48.0, 3: 55.0})]
    print(temperature_list)

if __name__ == '__main__':
    main()
```